Analysis FreeRTOS on STM32F411

1. Introduction

FreeRTOS is a real-time kernel (or real-time scheduler) on top of which embedded applications can be built to meet their hard real-time requirements. It allows applications to be organized as a collection of independent threads of execution. On a processor that has only one core, only a single thread can be executed at any one time.

The kernel decides which thread should be executing by examining the priority assigned to each thread by the application designer. In the simplest case, the application designer could assign higher priorities to threads that implement hard real-time requirements, and lower priorities to threads that implement soft real-time requirements.

Note. In FreeRTOS, each thread of execution is called a 'task'.

Free RTOS present you an analysis of

1.1. FreeRTOS Features

FreeRTOS has the following standard features:

- Pre-emptive or co-operative operation
- Very flexible task priority assignment
- Flexible, fast and light weight task notification mechanism
- Queues
- Binary semaphores
- Counting semaphores
- Mutexes
- Recursive Mutexes
- Software timers
- Event groups
- Tick hook functions
- Idle hook functions
- Stack overflow checking
- Trace recording
- Task run-time statistics gathering

- Optional commercial licensing and support
- Full interrupt nesting model (for some architectures)
- A tick-less capability for extreme low power applications
- Software managed interrupt stack when appropriate (this can help save RAM)

2. Objective

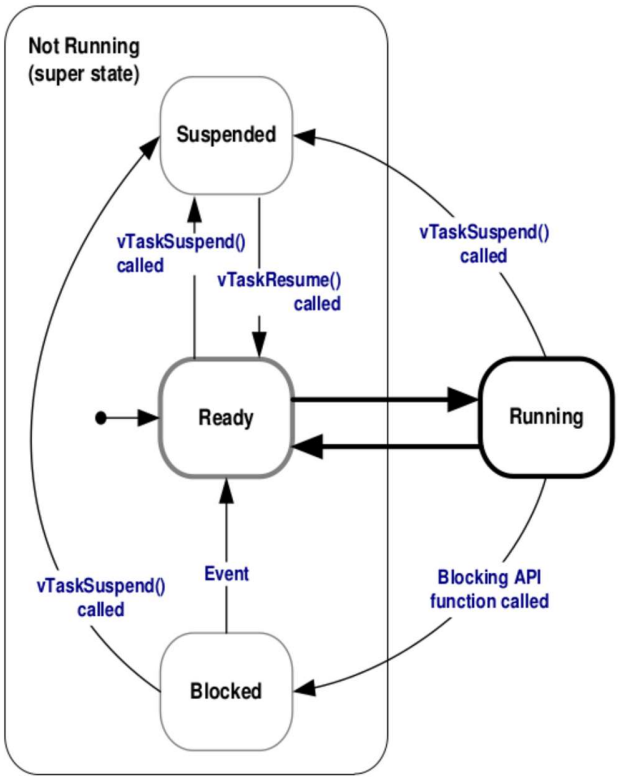
FreeRTOS provides a RTOS safety and strong to accomplish with the needs of project. Besides STM32Cube IDE has an interface to configure FreeRTOS on the board.

Once we choose FreeRTOS as our RTOS for the project, now we must understand how works the execution of task.

This document presents the functions used in the project.

3. Analysis of functions

Once we choose FreeRTOS as the RTOS for the project, now we must understand how works the execution of tasks.



3.1. Task delay until

The parameters to vTaskDelayUntil() specify, instead, the exact tick count value at which the calling task should be moved from the Blocked state into the Ready state. vTaskDelayUntil() is the API function that should be used when a fixed execution period is required (where you want your task to execute periodically with a fixed frequency), as the time at which the calling task is unblocked is absolute, rather than relative to when the function was called (as is the case with vTaskDelay()).

Once we choose FreeRTOS as the RTOS for the project, now we must understand how works the execution of features and tasks.

3.1.1. Task02 toggle and Task03 ON/OFF

Code

```
/* USER CODE END Header_StartTask02. osPriorityRealtime3 */
void StartTask02(void *argument)
{
    TickType_t xLastWakeTime;
    const TickType_t xDelay500ms = pdMS_TO_TICKS( 70 );

    xLastWakeTime = xTaskGetTickCount();

  for(;;)
  {
      HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_14);

      vTaskDelayUntil( &xLastWakeTime, xDelay500ms );

      //HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_14);
  }
}

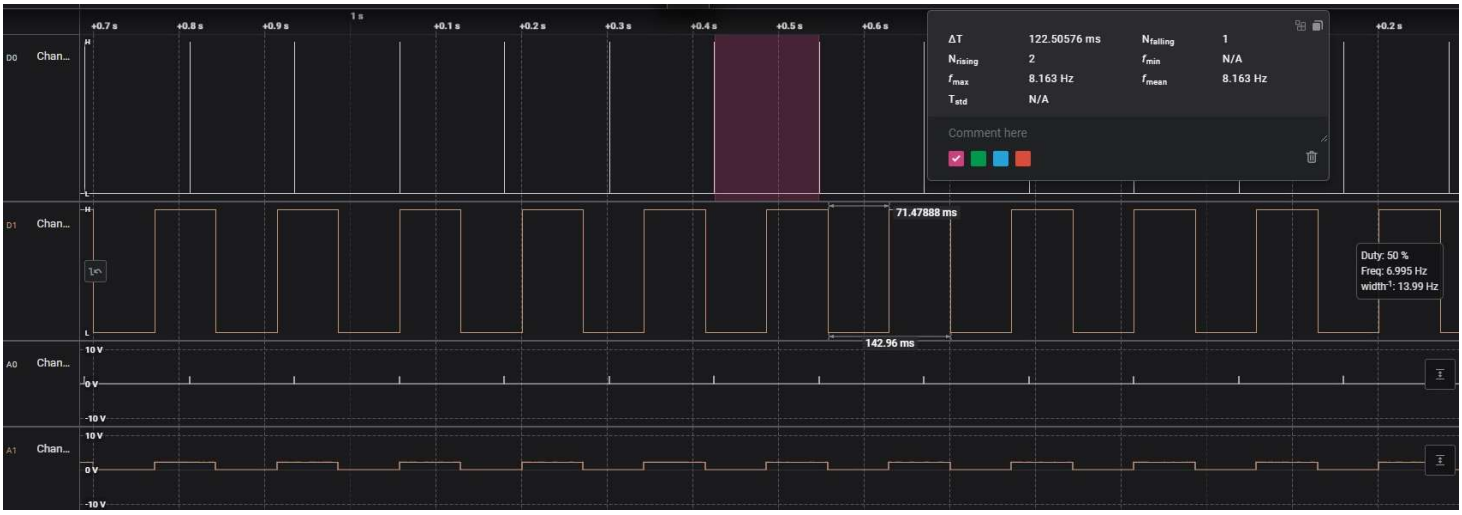
/* USER CODE END Header_StartTask03. osPriorityRealtime1 */
void StartTask03(void *argument)
{
    TickType_t xLastWakeTime;
    const TickType_t xDelay50ms = pdMS_TO_TICKS( 120 );

    xLastWakeTime = xTaskGetTickCount();

  for(;;)
  {
      //HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
      HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, 0); // 0 turn ON

      vTaskDelayUntil( &xLastWakeTime, xDelay50ms );

      HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, 1); // 0 turn Off
      //HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
  }
}
```



3.1.2. Task02 ON/OFF and Task03 ON/OFF

3.1.2.1. Both tasks are performing one shot in OFF state

```
/* USER CODE END Header_StartTask02. osPriorityRealtime3 */
void StartTask02(void *argument)
{
    TickType_t xLastWakeTime;
    const TickType_t xDelay70ms = pdMS_TO_TICKS( 70 );

    xLastWakeTime = xTaskGetTickCount();

  for(;;)
  {
      //HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_14);
      HAL_GPIO_WritePin(GPIOC, GPIO_PIN_14, 0); // 0 turn ON

      vTaskDelayUntil( &xLastWakeTime, xDelay70ms );

      HAL_GPIO_WritePin(GPIOC, GPIO_PIN_14, 1); // 0 turn Off
      //HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_14);
  }
}

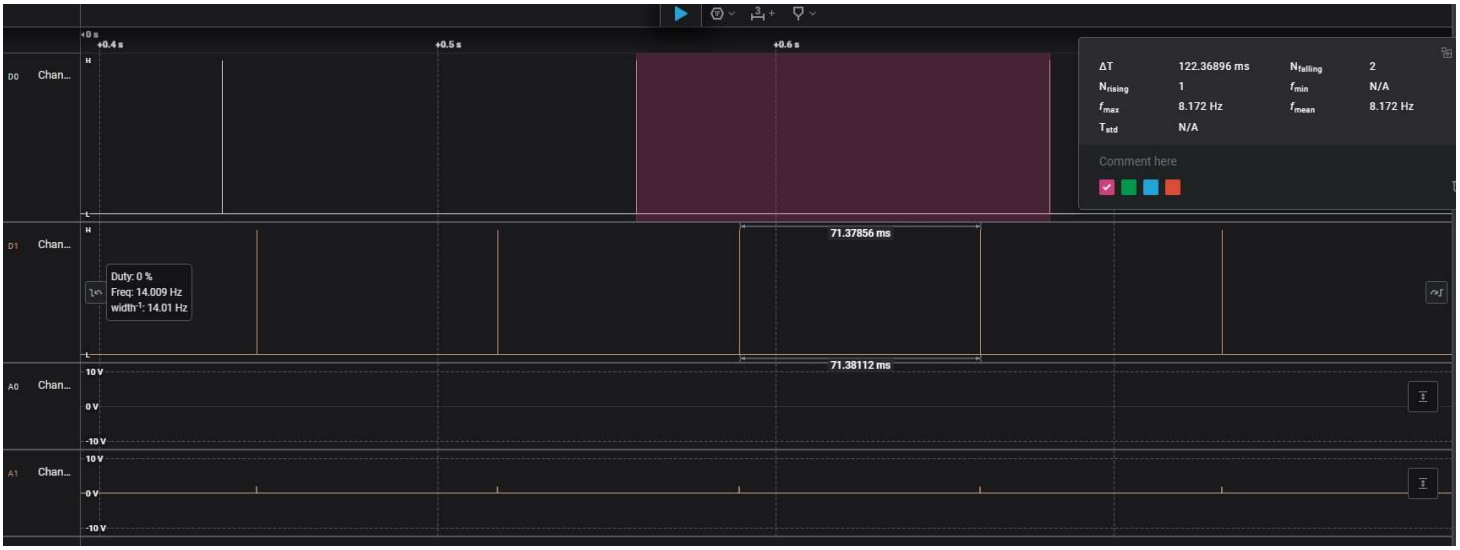
/* USER CODE END Header_StartTask03. osPriorityRealtime1 */
void StartTask03(void *argument)
{
    TickType_t xLastWakeTime;
    const TickType_t xDelay120ms = pdMS_TO_TICKS( 120 );

    xLastWakeTime = xTaskGetTickCount();

  for(;;)
  {
      //HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
      HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, 0); // 0 turn ON

      vTaskDelayUntil( &xLastWakeTime, xDelay120ms );

      HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, 1); // 0 turn Off
      //HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
  }
}
```



Task02 has a priority = Realtime3 (higher priority than Task03)

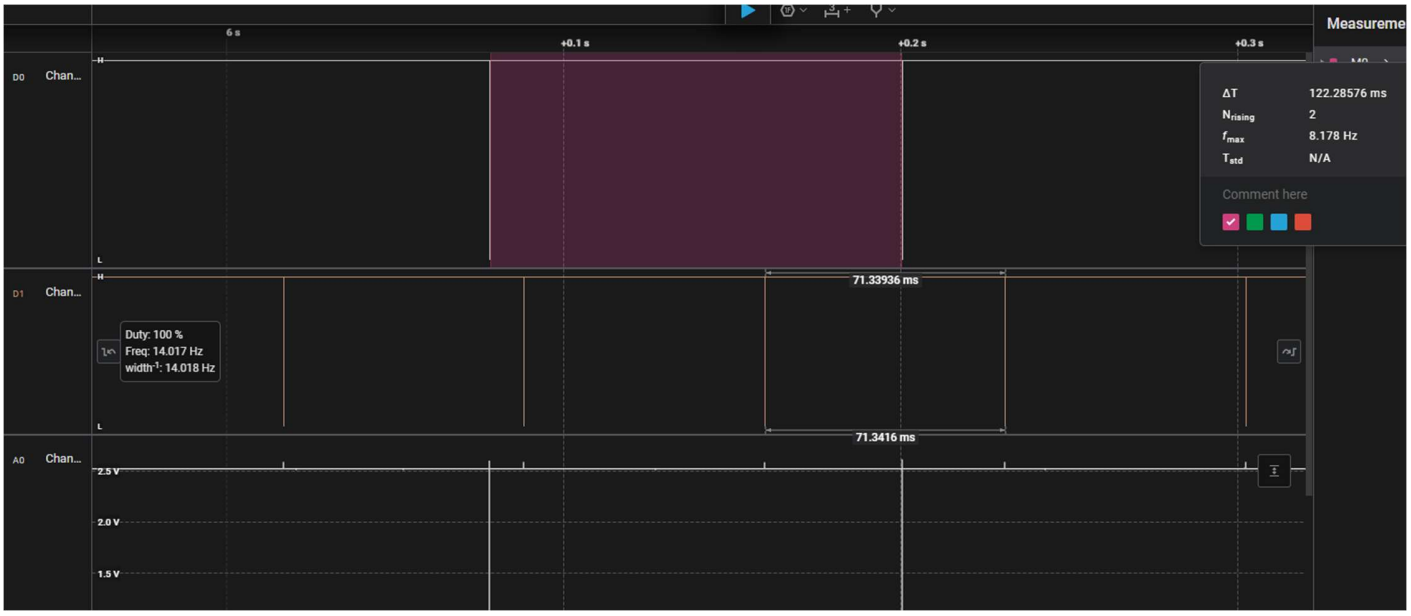
Task03 has a priority = Realtime1

FreeRtos is setting as PREEMPTIVE

When Task02 is triggered, Task02 sets OFF the gpio, then moves to Blocked state, here means the scheduler has again to choose a task to enter the running state, therefore Task03 enters to running state and sets OFF gpio, then moves to Blocked state. At 70ms Task02 sets ON gpio, but immediately Task02 sets OFF gpio (the next line in for cycle).

3.1.2.2. Both tasks are performing one shot in ON state.

```
300 void StartTask02(void *argument)
301 {
302     TickType_t xLastWakeTime;
303     const TickType_t xDelay70ms = pdMS_TO_TICKS( 70 );
304
305     xLastWakeTime = xTaskGetTickCount();
306
307   for(;;)
308   {
309       //HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_14);
310       HAL_GPIO_WritePin(GPIOC, GPIO_PIN_14, 1); // 0 turn ON
311
312       vTaskDelayUntil( &xLastWakeTime, xDelay70ms );
313
314       HAL_GPIO_WritePin(GPIOC, GPIO_PIN_14, 0); // 0 turn Off
315       //HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_14);
316   }
317 }
318
319
320 /* USER CODE END Header_StartTask03 */
321 void StartTask03(void *argument)
322 {
323     TickType_t xLastWakeTime;
324     const TickType_t xDelay120ms = pdMS_TO_TICKS( 120 );
325
326     xLastWakeTime = xTaskGetTickCount();
327
328   for(;;)
329   {
330       //HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
331       HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, 1); // 0 turn ON
332
333       vTaskDelayUntil( &xLastWakeTime, xDelay120ms ); //Goes to blocked state, but GPIO keeps ON
334                                                       //GPIO goes to off until pass 120ms
335
336       HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, 0); // 0 turn Off
337   }
338 }
```



Task03 has a priority = High (higher priority than Task02)

Task02 has a priority = AboveNormal

FreeRtos is setting as PREEMPTIVE

When Task02 is triggered, Task02 sets ON the gpio, then moves to Blocked state, here means the scheduler has again to choose a task to enter the running state, therefore Task03 enters to running state and sets ON gpio, then moves to Blocked state. At 120ms Task02 sets OFF gpio, but immediately Task02 sets ON gpio (the next line in for cycle).

3.2. Queue

FIFO

3.2.1. Receiving Data from multiple sources

xQueuSend()

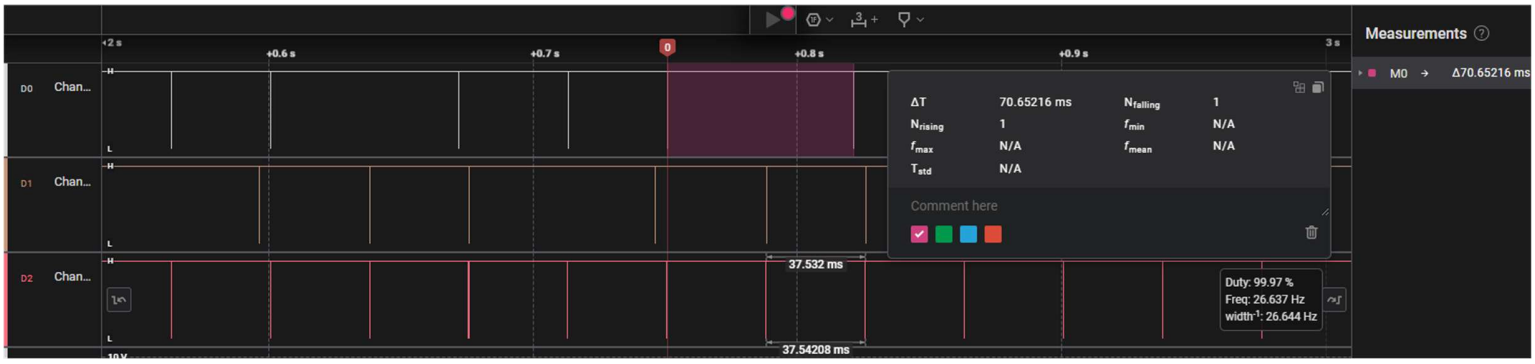
3.2.2. Receiving Data from Multiple Sources

Example 11

The sending tasks have the higher priority, so the queue will normally be full. This is because, as soon as the receiving task removes an item from the queue, it is pre-empted by one of the sending tasks which then immediately re-fills the queue. The sending task then re-enters the Blocked state to wait for space to become available on the queue again.

The receiving task has the lowest priority, so it will run only when both sending tasks are in the Blocked state. The sending tasks will enter the Blocked state only when the queue is full, so the receiving task will execute only when the queue is already full. Therefore, it always expects to receive data even when it does not specify a block time. In IDE for receive task was added delay = 120ms, but the task trigger when sender task are block, happens every 37ms

GPIO_PIN_15 = green led, blue wave, D2 Task3 (receive)
GPIO_PIN_14 = yellow led, geen wave, D1 Task2 (send) -70ms
GPIO_PIN_13 = blue led, yellow wave, D0 Task1 (send) -70ms

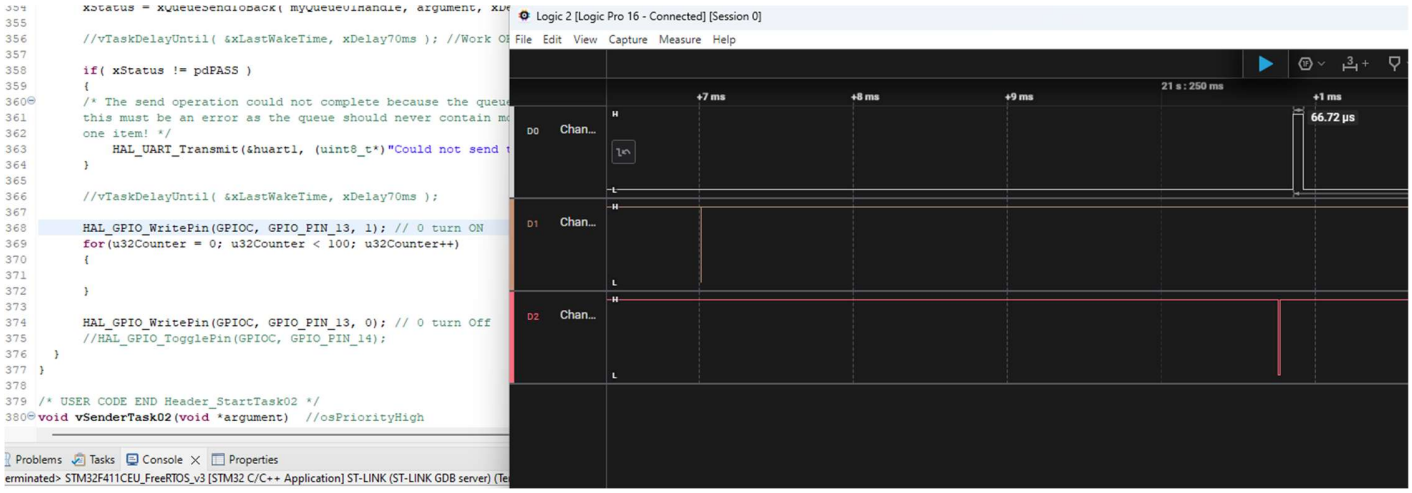


Seems when task changes led On to Off (just a short time before moves to running state).

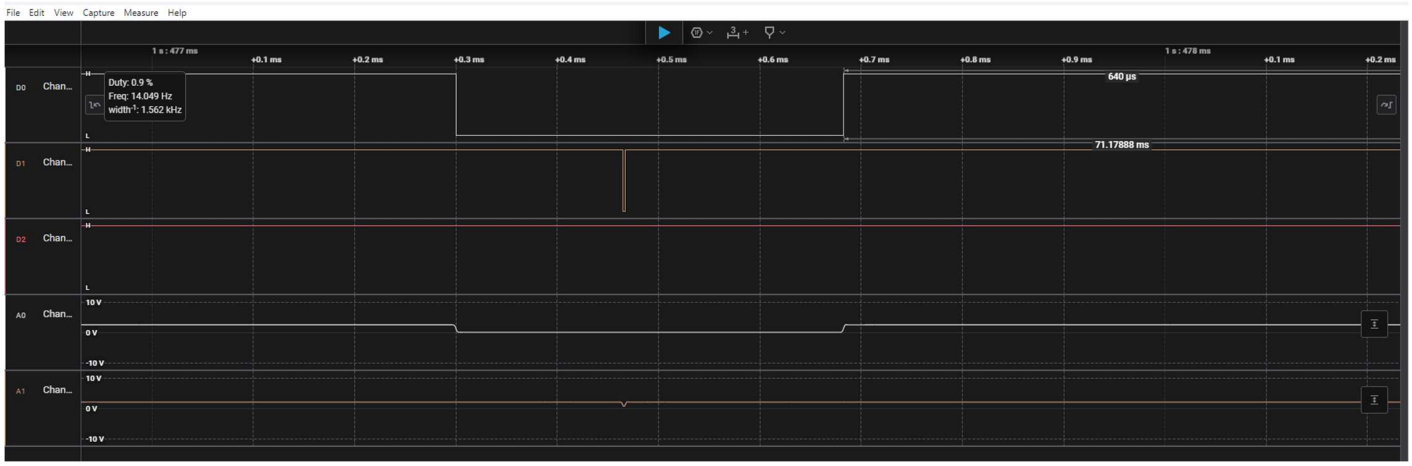
The sender tasks never (if is more than 70ms status = noPASS) run more than 70ms, sometimes is before. This is because the task should be kept in the Blocked state to wait for space to become available on the queue if the queue is already full. A block time is specified because the sending tasks have a higher priority than the receiving task so the queue is expected to become full. The receiving task will remove items from the queue when both sending tasks are in the Blocked state.

3.2.3. Check time(ms) takes a task, with Queue environment.

In Task1 (sender 1) 66us the amount of time, which takes perform a “for cycle” of 100 cycles.



In Task1 (sender 1). 640us the amount of time, which takes perform a “for cycle” of 1000 cycles.



I can consider a computer time of 1ms for a task using FreeRTOS.

3.2.4. To consider a mailbox

Using a Queue to Create a Mailbox

There is no consensus on terminology within the embedded community, and ‘mailbox’ will mean different things in different RTOSes. In this book the term mailbox is used to refer to a queue that has a length of one. A queue may get described as a mailbox because of the way it is used in the application, rather than because it has a functional difference to a queue:

 A queue is used to send data from one task to another task, or from an interrupt service routine to a task. The sender places an item in the queue, and the receiver removes the item from the queue. The data passes through the queue from the sender to the receiver.

 A mailbox is used to hold data that can be read by any task, or any interrupt service routine. The data does not pass through the mailbox, but instead remains in the mailbox until it is overwritten. The sender overwrites the value in the mailbox. The receiver reads the value from the mailbox but does not remove the value from the mailbox.

Will be use queue as 3.2.2

4. Anexos

4.1. Toolchain

5. Historial de revisión

Revisión	Nombre	Descripción	Fecha
1.0	Issac Jiménez	First Draft	17/11/2024

6. Referencias

[1] Lyl M. Ciganda, “Electrodos para medir pH (Junio 2004)”, XIII Seminario de Ing. Biomédica 2004 Facultades de Medicina e Ingeniería – Universidad de la República Oriental del Uruguay. 161204_Mastering_the_FreeRTOS_Real_Time_Kernel-A_Hands-On_Tutorial_Guide

https://atlas-scientific.com/